JAVA

ACTIVIDAD 7

1. Creamos los archivos sin nada solo el texto que no se auto genera: 3
a. Cliente.java: 3
b. Detalle.java: 3
c. ItemPedido.java 3
d. Pedido.java: 4
e. Principal.java: 4
2. Una vez hecho el codigo que no podemos autogenerar seguimos con el que si podemos. 5
a. Cliente.java: 5
b. Hacemos lo mismo con el resto de archivos. 9
3. Por último pasamo a .jar: 12

1. **Creamos los archivos sin nada solo el texto que no se auto genera:**

a. Cliente.java:

```
Cliente.java 3
src > Cliente.java > Cliente
1  public class Cliente {
2      private int idCliente;
3      private String nombre;
4      private String email;
5
6  }
7
```

b. Detalle.java:

```
Cliente.java 3   Detalle.java 3
src > Detalle.java > ...
1  public class Detalle {
2      private String color;
3      private String talla;
4      private double pesoGramos;
5  }
6
```

c. ItemPedido.java

```
Cliente.java 3   Detalle.java 3   Item
src > ItemPedido.java > ItemPedido > cantid
1  public class ItemPedido {
2      private Producto producto;
3      private int cantidad;
4  }
5
```

d. Pedido.java:

```
Cliente.java 3 ●   Detalle.java 3   ItemPedido.jav
src > Pedido.java > Pedido > fecha
1  import java.util.ArrayList;
2  import java.util.List;
3
4  public class Pedido {
5      private int idPedido;
6      private Cliente cliente;
7      private List<ItemPedido> items;
8      private String fecha;
9  }
10
```

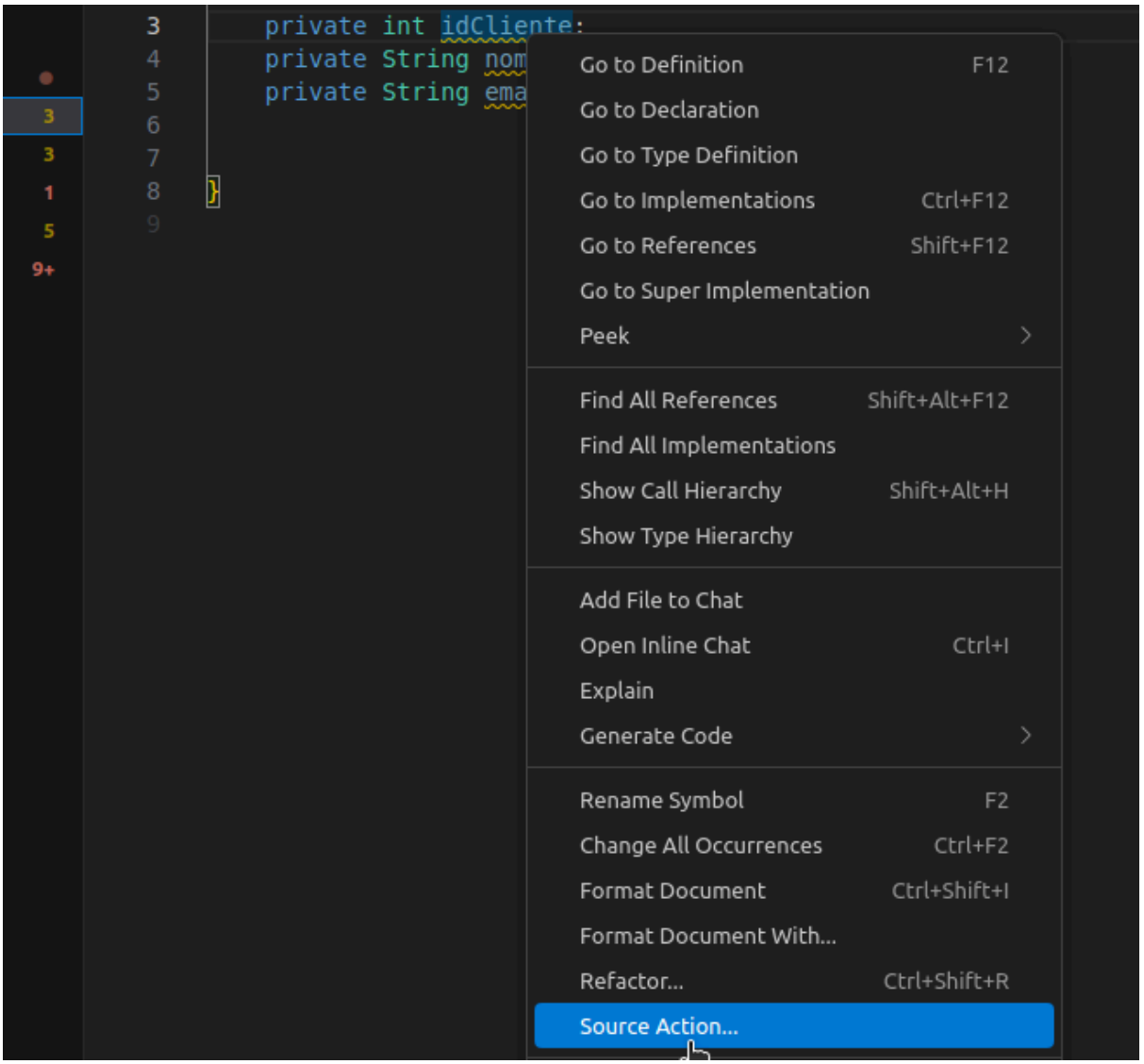
e. Principal.java:

```
Cliente.java 3 ●   Detalle.java 3   ItemPedido.java 1   Principal.java 9+ ×   Pedido.java 5 ●
src > Principal.java > Principal > main(String[])
1  public class Principal {
       Run | Debug
2      public static void main(String[] args) {
3          System.out.println(x: "--- Gestion de inventario y pedidos ---");
4          // 1. Creacion de objetos (Composicion y Agregación)
5
6          //Crear Detalle (Composición)
7          Detalle detalleCamiseta = new Detalle("Azul","L",250.0);
8          Detalle detalleLaptop = new Detalle("Gris Espacial","15 pulgadas",1500.0);
9
10         // Crear Productos (Contienen Detalle por Composicion)
11         Producto p1 = new Producto(101,"Camiseta Básica",19.99,"Ropa", detalleCamiseta);
12         Producto p2 = new Producto(205,"Laptop Pro",1250.00,"Electronica",detalleLaptop);
13         System.out.println(x: "\n --- Productos en el Inventario");
14         p1.mostrarInfo();
15         System.out.println(x: "------------------");
16         p2.mostrarInfor();
17
18         // Crear cliente (Agregación)
19         Cliente c1 = new Cliente(50,"Marta Ríos","marta.rios@mail.com");
20
21         // 2. Creacion  de Items y Pedidos
22
23         // Crear Items (agregacion: asocian Producto y cantidad)
24         ItemPedido item1 = new ItemPedido(p1,3); // 3 camisetas
25         ItemPedido item2 = new ItemPedido(p2,1); // 1 laptop
26
27         // Crear pedido
28         Pedido pedidoCLiente1 = new Pedido(1,c1,"2025-10-24");
29
30         // Agregar Items al pedido
31         pedidoCLiente1.agregarItem(item1);
32         pedidoCLiente1.agregarItem(item2);
33
34         // 3. mostrar resumen del pedido
35         pedidoCLiente1.mostrarResumen();
36
37         System.out.println(x: "--- Fin del proyecto ---");
38     }
39 }
```

03/10 println(String x) : void

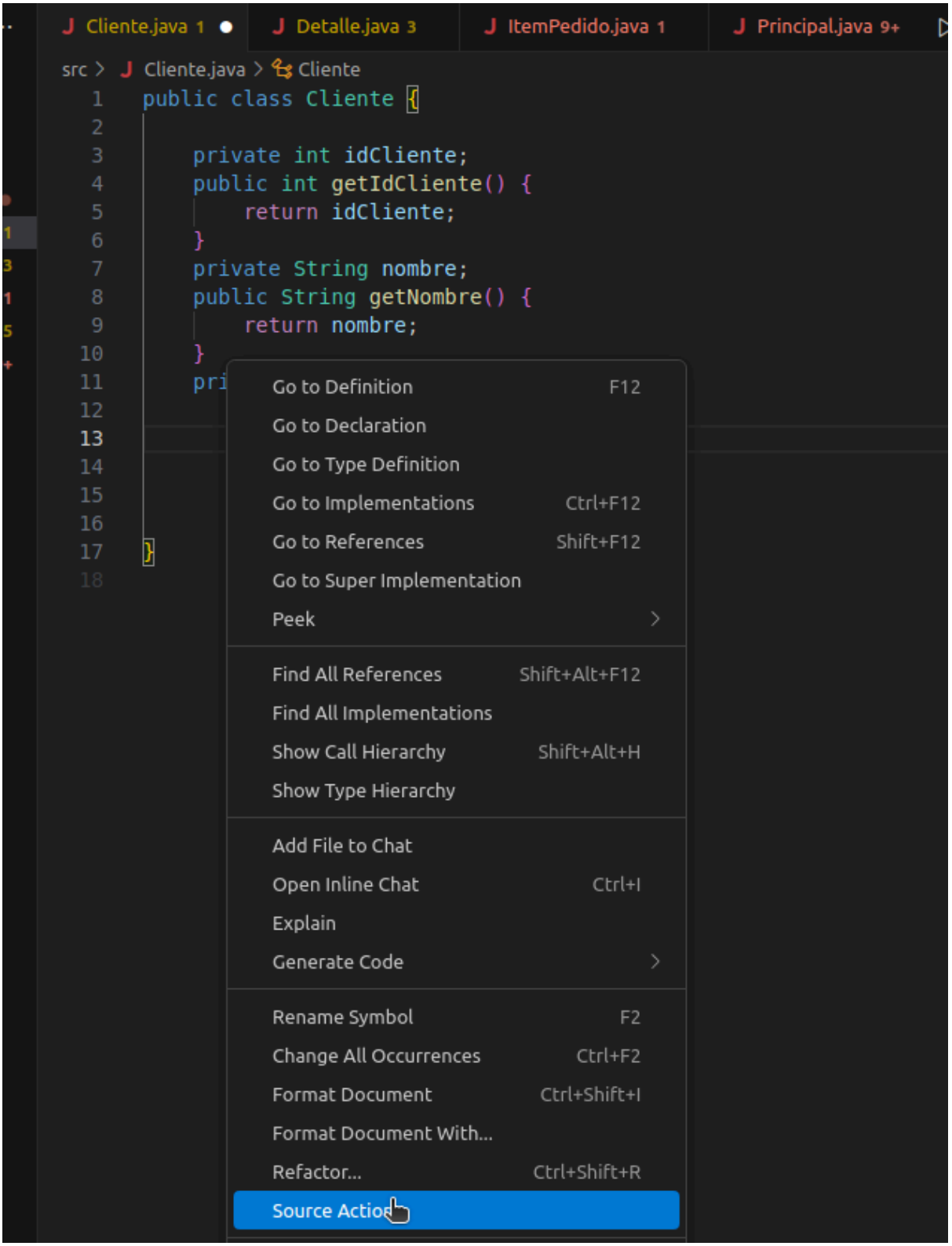
2. Una vez hecho el codigo que no podemos autogenerar seguimos con el que si podemos.

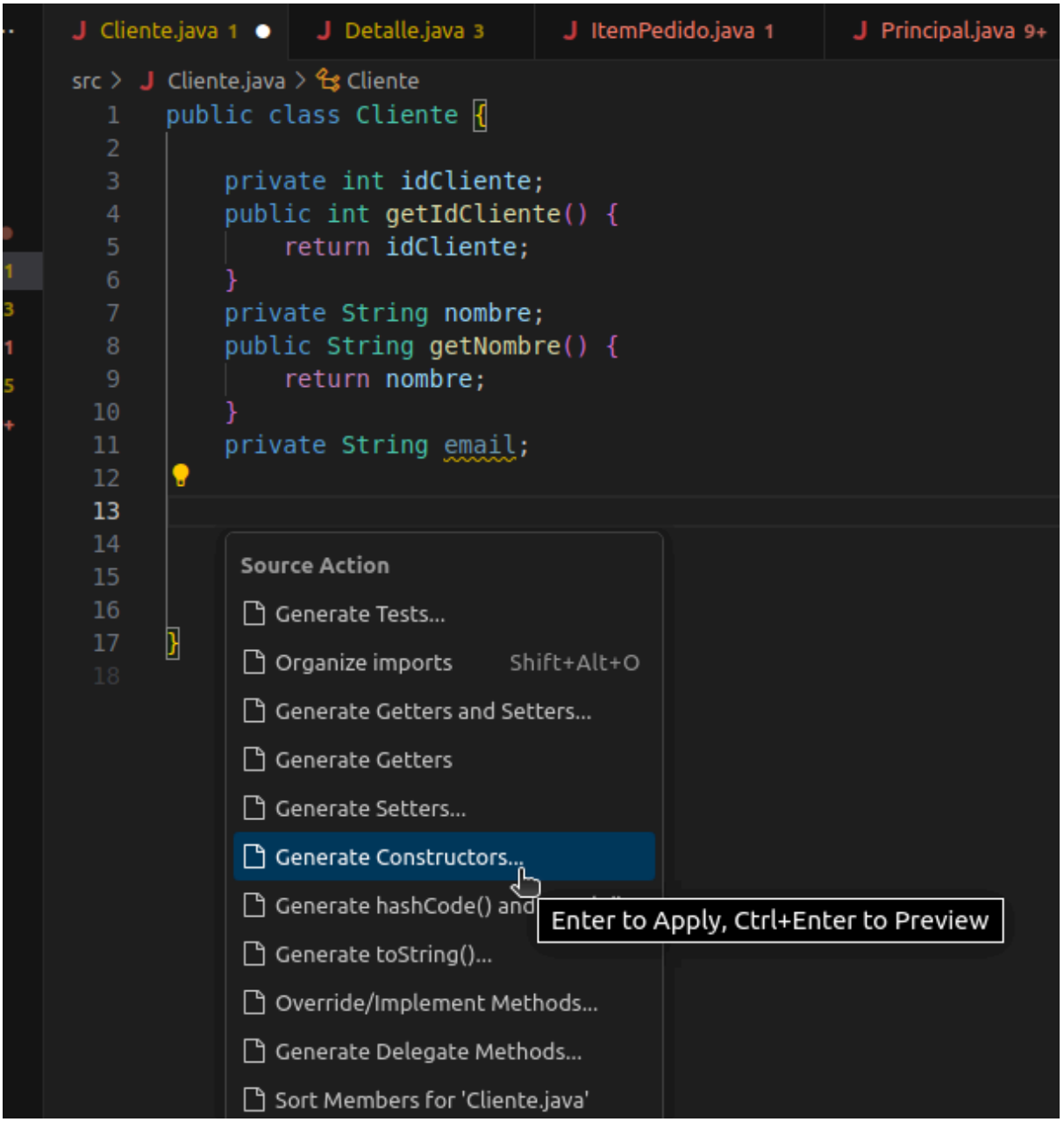
a. Cliente.java:



```
Cliente.java 2   Detalle.java 3   ItemPedido.java 1   Principal.java 9+
src > Cliente.java > Cliente > getIdCliente()
1  public class Cliente {
2
3      private int idCliente;
4      public int getIdCliente() {
5          return idCliente;
6      }
7      private String nombre;
8      private String email;
9
10
11 }
12
```

Seguimos con el resto de private y generamos el constructor:


Cliente.java 1
Detalle.java 3
ItemPedido.java 1
Principal.java 9+
src > Cliente.java > Cliente
public class Cliente {
private int idCliente;
public int getIdCliente() {
return idCliente;
}
private String nombre;
public String getNombre() {
return nombre;
}
pri
}
Go to Definition F12
Go to Declaration
Go to Type Definition
Go to Implementations Ctrl+F12
Go to References Shift+F12
Go to Super Implementation
Peek
Find All References Shift+Alt+F12
Find All Implementations
Show Call Hierarchy Shift+Alt+H
Show Type Hierarchy
Add File to Chat
Open Inline Chat Ctrl+I
Explain
Generate Code
Rename Symbol F2
Change All Occurrences Ctrl+F2
Format Document Ctrl+Shift+I
Format Document With...
Refactor... Ctrl+Shift+R
Source Action



Quedaria asi:

```java
Cliente.java > Cliente > mostrarInfo()
public class Cliente {

    private int idCliente;
    public int getIdCliente() {
        return idCliente;
    }
    private String nombre;
    public String getNombre() {
        return nombre;
    }
    private String email;
    public Cliente(int idCliente, String nombre, String email) {
        this.idCliente = idCliente;
        this.nombre = nombre;
        this.email = email;
    }

    public void mostrarInfo() {
        System.out.println("Cliente Id " + idCliente + ": " + nombre + " (" + email + ")");
    }

}
```

b. Hacemos lo mismo con el resto de archivos.

Detalle.java:

```java
EXPLORER
ACTIVIDAD 7
.vscode
bin
lib
src
Cliente.java
Detalle.java
ItemPedido.java 1
Pedido.java 5
Principal.java 8
README.md

src > Detalle.java > Detalle > mostrarDetalleFisico()
public class Detalle {
    private String color;
    private String talla;
    private double pesoGramos;
    public Detalle(String color, String talla, double pesoGramos
        this.color = color;
        this.talla = talla;
        this.pesoGramos = pesoGramos;
    }
    public void mostrarDetalleFisico() {
        System.out.println("   -Color: " + color);
        System.out.println("   -Talla/Tamaño: " + talla);
        System.out.println("   -Peso: " + pesoGramos + "g");
    }
}
```

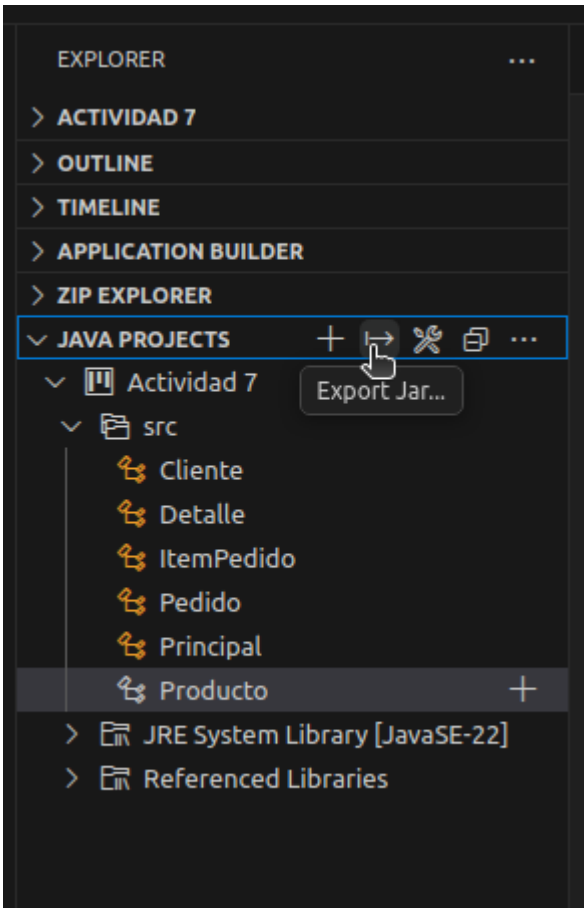
Itempedido.java:

```java
public class ItemPedido {
    private Producto producto;
    private int cantidad;

    public ItemPedido(Producto producto, int cantidad) {
        this.producto = producto;
        this.cantidad = cantidad;
    }
    public double calcularSubtotal(){
        return producto.getPrecio() * cantidad;
    }
    public void mostrarDetalle() {
        System.out.print("  -" + cantidad + " x ");
        System.out.print(producto.getNombre() + " (ID: " + producto.getIdProducto() + ")");
        System.out.println(" -> Subtotal: $" + String.format(format: "%.2f", calcularSubtotal()));
    }
}
```

Pedido.java:

```java
public class Pedido {

    public Pedido(int idPedido, Cliente cliente, String fecha) {
        this.idPedido = idPedido;
        this.cliente = cliente;
        this.fecha = fecha;
        this.items = new ArrayList<>();
    }

    public void agregarItem(ItemPedido item) {
        items.add(item);
    }

    public double calcularTotal() {
        double total = 0;
        for (ItemPedido item : items) {
            total += item.calcularSubtotal();
        }
        return total;
    }

    public void mostrarResumen() {
        System.out.println("\n*** RESUMEN DE PEDIDO ID " + idPedido + " ***");
        System.out.println(" Fecha " + fecha);
        System.out.println(x: "Cliente: ");
        cliente.mostrarInfo();
        System.out.println(x: "\n Detalle de ítems:");
        for (ItemPedido item : items) {
            item.mostrarDetalle();
        }
        System.out.println("\n TOTAL A PAGAR: $" + String.format(format: "%.2f", calcularTotal()));
        System.out.println(x: "***************************");
    }

}
```

Producto.java:

```java
public class Producto {
    private int idProducto;
    private String nombre;
    private double precio;
    private String tipo;
    private Detalle detalle; // 👈 Asociación con la clase Detalle

    // Constructor con Detalle
    public Producto(int idProducto, String nombre, double precio, String tipo, Detalle detalle) {
        this.idProducto = idProducto;
        this.nombre = nombre;
        this.precio = precio;
        this.tipo = tipo;
        this.detalle = detalle;
    }

    // Getters y setters
    public int getIdProducto() { return idProducto; }
    public void setIdProducto(int idProducto) { this.idProducto = idProducto; }

    public String getNombre() { return nombre; }
    public void setNombre(String nombre) { this.nombre = nombre; }

    public double getPrecio() { return precio; }
    public void setPrecio(double precio) { this.precio = precio; }

    public String getTipo() { return tipo; }
    public void setTipo(String tipo) { this.tipo = tipo; }

    public Detalle getDetalle() { return detalle; }
    public void setDetalle(Detalle detalle) { this.detalle = detalle; }

    // Método para mostrar información completa del producto
    public void mostrarProducto() {
        System.out.println("ID Producto: " + idProducto);
        System.out.println("Nombre: " + nombre);
        System.out.println("Precio: " + precio + " €");
        System.out.println("Tipo: " + tipo);
        System.out.println(x: "Detalle físico:");
        detalle.mostrarDetalleFisico(); // 👈 Usa el método de la clase Detalle
    }

    public void mostrarInfo() {
        // TODO Auto-generated method stub
        throw new UnsupportedOperationException(message: "Unimplemented method 'mostrarInfor'");
    }
}
```

3. Por último pasamo a .jar: